# Certifiable Traffic Engineering for Software-Defined Networks: Exact Bottleneck Bounds, Hybrid Ant-Colony Selection, and Evaluation on Real Research Backbones

João Paulo Versiani Ladeia, and Nilton Alves Maia

***Abstract*—Traffic-engineering (TE) heuristics for software-defined networks are usually validated against shortest-path baselines on small synthetic graphs, and their quality claims are rarely certified. We formulate multi-flow QoS routing with hard capacity and end-to-end delay constraints and a min–max link-utilization objective, and solve it exactly as a binary multi-commodity flow (ILP) solved lexicographically — first the bottleneck $Z$, then a delay tie-break — so the reported utilization bounds are certified. The same ILP certifies the optimality gap of a two-stage hybrid (Ant-Colony candidate pool + Genetic Algorithm or Differential Evolution selection) within its own search space. Over five load factors, twenty replications and five instances (the original 52-node study network, two synthetic graphs, and two public backbones — GEANT and a German research network — with real coordinates, distances and fiber-derived latencies), the certificate changes the reading of the results: at $1.25\times$ session growth ($1.13\times$ aggregate traffic) on the sparse instance, *no* routing can keep every link below 98.4% utilization, and at $1.5\times$ ($1.26\times$ traffic) a feasible routing still exists with only $1.6\%$ headroom — yet the stochastic selector attains it in 20% of replications. Seeding the candidate pool with the constructive CSPF solution guarantees the hybrid is never worse than that baseline by construction, and removes a seed-dependence that halved its feasibility. On GEANT the constructive baseline itself is infeasible ($1.52\times$ capacity) and the hybrid remains feasible in 85% of replications. Packet-level simulation (queues, shared per-pair capacities and admission-drop loss identical for all methods) shows delay-sensitive flows mapped to the URLLC class held at 29 ms and 0.0% loss under $2\times$ background on the real backbone, against 182 ms and 2.8% for shortest-path routing — logical differentiation under shared capacity, not guaranteed isolation. A computed data-plane accounting shows SRv6 segment lists reduce per-flow forwarding state by $2.1$–$4.1\times$, with the reduction tracking mean path length. We position exact certification as the instrument that makes metaheuristic TE auditable, and delimit honestly where stochastic selection degrades.**

***Index Terms*—Software-defined networking, traffic engineering, QoS routing, ant colony optimization, genetic algorithm, differential evolution, integer programming, optimality certification, network slicing, SRv6, deep reinforcement learning.**

## I. Introduction

OPERATORS face a structural tension: traffic grows faster than revenue, and capacity expansion is a recurring capital expense. Traffic engineering (TE) attacks this tension from the software side — it routes the traffic that exists more intelligently instead of adding links. Software-Defined Networking (SDN) supplies the missing control point: by decoupling the control plane and exposing programmable forwarding [1]–[3], an SDN controller can compute and install paths that satisfy per-application requirements instead of replicating shortest-path behavior.

This paper studies exactly that problem: given a topology with per-link capacity and delay, and a set of $L$ concurrent applications with minimum bandwidth, maximum delay and priority requirements, compute a route for every application that maximizes admission quality under QoS constraints. We make the following contributions, each of which responds to a concrete methodological gap in the literature on heuristic SDN-TE:

1) **Revised formulation.** We analyze the scoring-function model commonly used in heuristic SDN-TE, identify its degeneracies (constant terms under full admission; capacity counted against a flow cardinality), and reformulate the problem with *hard* capacity and delay constraints and an explicit min–max utilization balancing objective (Section III).
2) **Exact certification.** We solve the revised model exactly as a binary multi-commodity flow ILP (lexicographic: bottleneck first, delay tie-break) and the scoring model exactly over the candidate pool. The exact solutions (i) certify the optimality gap of the pool-augmented hybrid within its own search space, and (ii) compute *exact bottleneck lower bounds* — the minimum achievable maximum link utilization — information that stochastic heuristics do not provide and that exposes how far greedy and learning-based baselines sit from the provable optimum (Section VIII-A).
3) **Validation on real backbones.** Prior hybrid-TE evaluations are confined to small synthetic graphs. We repeat the full protocol on two *real* public backbones (real coordinates, distances and fiber-derived latencies; synthetic demands and calibrated capacities) and identify a regime — in which the constructive baseline itself becomes infeasible — that the synthetic 52-node instances do not expose (Sections VIII-B and VIII-C).
4) **Statistical evaluation.** We evaluate six routing methods on five instances across five load factors and twenty

Manuscript received XXX; revised XXX. The authors are with the Graduate Program in Computational Modeling and Systems (PPGMCS), Universidade Estadual de Montes Claros, Montes Claros, Brazil.

randomized replications, with 95% confidence intervals and Wilcoxon signed-rank tests, identifying the load regime in which the hybrid strictly dominates greedy and learning-based baselines (Section VIII-B).

5) **Computed SRv6 state accounting.** We map the optimized paths to SRv6 segment lists [16]–[18] and *compute* the per-flow forwarding state on all instances, quantifying the reduction with respect to per-flow OpenFlow rules and bounding the SRH overhead (Section V).
6) **Logical differentiation evidence.** We connect the demand profile to 3GPP service classes [21], [22] and measure, in packet-level simulation on a real backbone, the stability of delay-sensitive flows (mapped to the URLLC class) under growing background load — comparing optimized against shortest-path routing (Section VI).
7) **Learning-based baseline.** We implement a tabular Q-learning routing agent with the same state–action structure as DRL routers and benchmark it under the same protocol, grounding the DRL discussion in measured numbers rather than assertions (Section VIII-H).

The remainder is organized as follows. Section II reviews related work. Section III presents the system model. Section IV describes the algorithms. Sections V and VI present the SRv6 mapping and slice differentiation. Section VII details the experimental methodology, Section VIII reports results, Section IX discusses threats to validity, and Section X concludes.

## II. Related Work

**Traffic engineering.** TE principles and IGP-metric optimization are established [8], [9], and production backbones perform centralized TE at scale [10]. Exact link-based MCF formulations of the kind we use for certification are the classical tool of that line — our contribution is not the ILP itself but the systematic use of its certificates to audit a heuristic pipeline.

**Heuristic and hybrid routing.** Ant-based routing originates with AntNet [5], [7], and ACO has been applied to QoS routing repeatedly since. Two-stage "generate-and-select" hybrids — a constructive stage feeding an evolutionary selector — are the standard memetic pattern; the present work instantiates it for SDN with an explicit admission-scoring objective and, critically, adds exact certification that prior hybrid-routing studies omit.

**Learning-based routing.** DRL for SDN routing has progressed from experience-driven proposals [13], [14] to evaluated traffic-engineering agents [15]. Reported strengths are online adaptivity and traffic-matrix generalization; reported costs are large training campaigns and the sim-to-real gap. Our experiments (Section VIII-H) add a controlled, constraint-explicit comparison on identical instances.

**Programmable data planes.** OpenFlow [1] programs switches with per-flow rules; P4 [4] generalizes parsing and matching. Both expose the TCAM-pressure problem that motivates SRv6 [16]–[19]: the path state is carried in the packet, and only headends hold policy state. To our knowledge, the state reduction of routing-optimizer output under SRv6 has not been measured on fixed topologies with optimizer-produced paths; Section V reports it.

**Network slicing.** 3GPP defines slice/service types and QoS flows [22] anchored on the IMT-2020 usage scenarios [21]. Transport-side slice isolation is usually pursued with dedicated resources; we show that constraint-explicit routing yields measurable logical differentiation under shared capacity — priority flows degrade last and least — without physical partitioning (Section VI).

## III. System Model and Problem Formulation

The network is a directed graph $G = (V, E)$, $|V| = n$; link $(i, j) \in E$ has capacity $\mu_{ij}$ (Mbps) and one-way delay $c_{ij}$ (ms). A set of $L = 17$ applications must be routed concurrently; application $k$ requires nominal flow $\lambda_k$ (Mbps), tolerates end-to-end delay $h_k$ (ms) and carries priority $\delta_k$. Let $x_{ij}^k \in \{0, 1\}$ indicate whether the route of application $k$ uses link $(i, j)$.

### A. Original Scoring Model

The original model maximizes the weighted admission score

$$\mathrm{FO} \;=\; \sum_{k=1}^{L} \delta_k\, y_k + p_1 \sum_{k=1}^{L} y_k + p_2 \sum_{k=1}^{L} w_k + p_3 \sum_{k=1}^{L} t_k, \quad (1)$$

with $p_1$=3, $p_2$=2, $p_3$=1; $y_k$ marks an established connection, $w_k$ capacity-sufficient routes and $t_k$ delay-conforming routes. Two structural observations motivate the revised model: (i) under full admission every $y_k = 1$, so the first two sums are constant and the score reduces to $\mathrm{FO} = \Sigma\delta_k + 6L - 2\, n_{\mathrm{over}} - n_{\mathrm{dv}}$, where $n_{\mathrm{over}}$ counts overloaded *links* and $n_{\mathrm{dv}}$ counts delay-violating *flows*; (ii) the mixed cardinality (links vs. flows) makes weights comparable only by accident. The score saturates at $\mathrm{FO}^* = \Sigma\delta_k + 6L$ (= 132 for this demand profile): any fully feasible solution attains it, and the score cannot discriminate among solutions with different load balance.

### B. Revised Model: Hard Constraints and Balancing

We therefore evaluate solutions on a revised model that separates feasibility from quality:

$$\text{routing (per flow):} \quad \sum_j x_{ij}^k - \sum_j x_{ji}^k = \begin{cases} 1 & i = s_k \\ -1 & i = t_k \\ 0 & \text{otherwise} \end{cases} \qquad \forall k,\, i, \quad (2)$$

$$\text{delay (hard):} \quad \sum_{(i,j)\in E} c_{ij}\, x_{ij}^k \;\le\; h_k \qquad \forall k, \quad (3)$$

$$\text{balance (objective):} \quad \min\; Z, \quad Z \;\ge\; \frac{1}{\mu_{ij}} \sum_{k=1}^{L} \lambda_k\, x_{ij}^k \quad \forall (i, j). \quad (4)$$

with $\sum_k \lambda_k x_{ij}^k + \sum_k \lambda_k x_{ji}^k \le \mu_{uv}$ for each undirected pair $\{u, v\}$: the two directions of a link share one capacity, matching bidirectional links carrying traffic in both senses. The model is solved *lexicographically*: first minimize $Z$ alone (certifying the exact bottleneck bound), then minimize total route delay subject to $Z \le Z^*$ as a deterministic tie-break — a weighted combination would not certify the minimum

of $Z$. A solution is *feasible* iff (**??**) and (3) hold for all links and flows; among feasible solutions we prefer minimum maximum utilization $Z$. When (**??**)–(3) admit no solution at all (certified by the exact model), every heuristic output violates some constraint, and the natural metric becomes the number of violations it inflicts.

## IV. Routing and Optimization Methods

### A. Stage 1: Candidate-Route Generation (ACO/AntNet)

For each application an ant colony builds source–destination walks with the transition probability

$$P(i,j) = \frac{[\tau_{ij}]^{\alpha}[\eta_{ij}]^{\beta}}{\sum_{l\in\mathcal{N}_i}[\tau_{il}]^{\alpha}[\eta_{il}]^{\beta}}, \tag{5}$$

repeated nodes are removed, links reinforce pheromone proportionally to the carried bandwidth, and evaporation rate $\rho$ maintains diversity. With 100 ants $\times$ 100 iterations this yields 362–1014 distinct candidate routes per topology (Table II).

### B. Stage 2: Route-Set Selection (ACO+GA / ACO+DE)

An individual is an integer vector selecting one pool route per application (each selection constrained to its application's block). The GA applies roulette selection (pressure 8), arithmetic crossover ($\gamma$=0.4) and Gaussian mutation; DE applies rand/1 mutation ($\beta \sim \mathcal{U}(0.2, 0.8)$) with binomial crossover ($p_{CR}$=0.2). Both maximize (1) with 300 generations over 50 individuals. We emphasize explicitly: the metaheuristics optimize the *original scoring model* (Model A, Section III-A) — the same objective as the original study — whereas the revised model (Model B, Section III-B) is used for evaluation and certification. The two are related but distinct problems; we do not claim that the selector optimizes Model B, and Section VIII-B quantifies the price of that choice (weaker load balancing than the exact Model B optimum). *Pool augmentation.* Two constructive augmentations make the selector's search space self-sufficient: (i) for load factors $f > 1$, each additional background flow receives $k$-shortest-delay candidate paths; (ii) for every flow, the CSPF constructive route is inserted into its block. Augmentation (ii) guarantees that the selector's optimum is never worse than CSPF — the CSPF assembly is inside the search space — while remaining free to improve on it whenever the pool holds a better combination. This closes the classical objection against generate-and-select designs: pool diversity, not the selector, becomes the binding constraint, and the augmentation pins the selector to at least the constructive baseline.

### C. Baselines

**Dijkstra.** Each application follows the geometric-shortest path (the metric of the original study) — the shortest-path IGP-like behavior; it ignores capacity by design and serves as the lower baseline, whereas CSPF below is the strong constructive baseline. **CSPF.** Constrained shortest path first: flows are admitted in priority order; links whose residual capacity is below the request are pruned; the delay-minimizing path on the pruned graph is selected; if no pruned path satisfies the delay budget the flow is admitted on an unconstrained path (honest best-effort, counted as a violation when it overloads links). **Q-learning** [25]. A tabular, per-flow, sequential learning agent with node-state/neighbor-action structure, $\epsilon$-greedy exploration, capacity-penalty shaping, and a capacity-pruned greedy rollout (Section VIII-H). **ILP (exact).** Two exact formulations close the methodological loop: (i) *ILP-P* maximizes (1) over the same pool explored by the GA/DE, certifying the metaheuristics' optimality gap; (ii) *ILP-L* solves the revised model (**??**)–(4) exactly as a binary multi-commodity flow with min–max utilization, certifying feasibility or infeasibility of each instance.

## V. SRv6 Data-Plane Mapping and Measured State Reduction

Per-flow OpenFlow programming stores one match–action rule per flow at every transit switch; the aggregate state is $\Phi_{\mathrm{OF}} = \sum_k (|p_k| - 1) \le K L_{\max}$ [1]. SRv6 [16]–[18] carries the path state in the packet: each node $v$ is assigned a Segment Identifier $\sigma(v) = \mathrm{LOC}(v)::\mathrm{FUNC}_{\mathrm{End}}$ [18], and the optimal route $p_k^* = (v_1^{(k)}, \ldots, v_{m_k}^{(k)})$ is encoded as the segment list

$$\mathcal{SL}_k = \mathrm{rev}\big[\sigma(v_2^{(k)}), \ldots, \sigma(v_{m_k}^{(k)})\big], \tag{6}$$

so that on arrival at $v_i^{(k)}$ the outer destination equals $\sigma(v_i^{(k)})$ and $\mathrm{SegmentsLeft} = m_k - i$; the flow-agnostic `End` function advances the segment and the egress decapsulates. Because the ACO stage removes repeated nodes, $p_k^*$ is a simple path and the SRH is well formed (no micro-loops). The two-stage optimizer thereby acts as a centralized Path Computation Element [20]: it emits for each flow a Segment Routing Policy $\Pi_k = \langle H_k, C_k, \mathcal{SL}_k \rangle$ [19] whose color $C_k$ carries the serving slice identifier; the controller programs only headends.

Under this mapping the per-node rule footprint is

$$\Phi_{\mathrm{SR}}(v) = \begin{cases} |\{k : v = v_1^{(k)}\}| & \text{if } v \text{ is a headend,} \\ 0 & \text{otherwise,} \end{cases} \tag{7}$$

i.e., $\mathcal{O}(K)$ globally and $\mathcal{O}(1)$ per transit node. Section VIII-G reports the *computed* state counts (an analytical accounting over the produced paths, not a hardware measurement) on all five instances, together with the SRH encapsulation overhead ($40 + 8 + 16\,|\mathcal{SL}_k|$ bytes per packet).

## VI. 5G/6G Slice Differentiation and SLA Guarantees

The 17 applications instantiate the three IMT-2020 service classes [21] with the standardized slice/service types of 3GPP TS 23.501 [22]: *URLLC* (SST = 2) gathers conversational voice and interactive control (apps 8, 9, 13, 15; $h_k = 150$ ms, maximal priority $\delta_k = 5$); *eMBB* (SST = 1) gathers high-definition video (apps 3, 4, 11, 12; 1.20–1.74 Mbps); *mMTC* (SST = 3) gathers telemetry and elastic transfer (apps 1, 2, 5, 6, 7, 10, 14, 16, 17; $h_k = 1000$ ms). Isolation follows from the constraint structure: the capacity constraint (**??**) is evaluated on the *aggregate* cross-slice offered load of each link, so a feasible solution bounds the interference any slice can exert on another; the priority term of (1) biases admission

TABLE I
APPLICATION DEMANDS (ALL INSTANCES): NOMINAL FLOW, DELAY BUDGET AND PRIORITY. CLASSES PER SECTION VI.

| Class | Apps | $\lambda_k$ (Mbps) | $h_k$ (ms) | $\delta_k$ |
|---|---|---|---|---|
| URLLC | 8, 9, 13, 15 | 0.512 | 150 | 5 |
| eMBB | 3, 4, 11, 12 | 1.20–1.74 | 150 | 0–5 |
| mMTC | 1, 2, 5, 6, 7, 10, 14, 16, 17 | 0.256 | 1000 | 0 |

toward URLLC; and the delay constraint binds each route to its slice budget. The result is deterministic, priority-ordered degradation — the priority slice degrades last and least — rather than the guaranteed isolation of a dedicated partition; we measure this distinction explicitly in Section VIII-F. Section VIII-F measures this directly: delay-sensitive (URLLC-mapped) flows keep their 150 ms experimental budget under growing background amplification with optimized routing — on both the study instance and the real backbone — and do not under shortest-path routing.

## VII. EXPERIMENTAL METHODOLOGY

### A. Topologies and Demands

We evaluate five instances (Fig. 1): the original 52-node study topology (*maia52*, 66 links, uniform 2 Mbps, delays 1–80 ms); two synthetic 52-node graphs with controlled structure (*waxman52*, a connected Waxman random geometric graph [6], 177 links; and *gridcore*, a 48-node peripheral grid plus a 4-node dense core, 96 links); and two *real* public backbones taken from the Internet Topology Zoo [11] and the SNDlib network-design library [12]: *geant2012* (37 nodes, 58 links, a pan-European research and education network spanning long-haul links up to 3219 km) and *germany50* (50 nodes, 88 links, a national research network dominated by regional links). For the two real backbones we recover link latency from the genuine great-circle distances shipped with those datasets using the fiber propagation constant $5\,\mu$s/km plus a fixed $0.5$ ms per-hop term, yielding 1.0–1.8 ms (regional) and 1.0–16.6 ms (long-haul) — i.e., realistic backbone latencies, two orders of magnitude below the synthetic range. The public GML exports carry no link-capacity attribute, so we apply one fixed, documented calibration to every instance: solve the exact min–max multi-commodity problem once with a generous capacity, read off the heaviest link load $L$, and set every link to $L/0.87$, which places each instance at a certified bottleneck utilization of $0.87$ at nominal load — the congestion point certified for the original study. This makes cross-instance comparison well posed and is reported as a limitation (Section IX). The 17-flow demand profile of Table I (total 9.70 Mbps) is applied to all five instances, with endpoints re-drawn per instance and connectivity guaranteed.

### B. Load Sweep and Replication

For $f \leq 1$ all flow demands are scaled by the load factor $f \in \{0.5, 0.75, 1.0\}$; for $f > 1$ (labeled $\{1.25, 1.5\}$) the nominal flows are kept at nominal rate and $\mathrm{round}((f-1)\cdot 20)$ additional concurrent background data flows (0.256 Mbps, $h_k$=1000 ms) are added on random connected endpoint pairs

TABLE II
TOPOLOGY INSTANCES AND ACO CANDIDATE-POOL SIZES (100 ANTS × 100 ITERATIONS, 20 SEEDS). *real* MARKS INSTANCES BUILT FROM PUBLIC BACKBONE DATASETS WITH GENUINE COORDINATES AND LINK DISTANCES.

| Instance | Type | Nodes | Links | Mean deg. | Pool size (routes) |
|---|---|---|---|---|---|
| maia52 | study | 52 | 66 | 2.54 | 362–420 |
| waxman52 | synthetic | 52 | 177 | 6.81 | 927–1014 |
| gridcore | synthetic | 52 | 96 | 3.69 | 706–780 |
| germany50 | *real* | 50 | 88 | 3.52 | 639–728 |
| geant2012 | *real* | 37 | 58 | 3.14 | 330–409 |

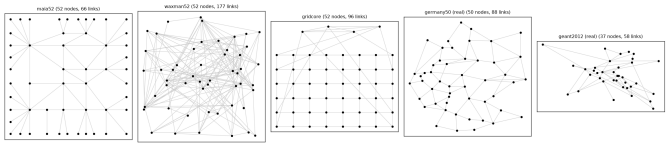


Fig. 1. Evaluated instances: the original 52-node study topology (maia52), two synthetic 52-node graphs (waxman52, gridcore) and two real public backbones (germany50, geant2012, *marked real*) built from genuine coordinates and link distances. All share the same 17-flow demand profile and the same calibrated congestion point.

— modeling network growth as new sessions rather than inflating existing flows beyond the physical link capacity, which would certify infeasibility trivially. The labels are session-growth parameters, *not* literal aggregate multipliers: the corresponding aggregate offered traffic is 1.13× ("1.25") and 1.26× ("1.5") the nominal, and we quote the labels throughout with this meaning. Stochastic methods (ACO+GA, ACO+DE, Q-learning) are replicated over 20 seeds per (topology, load); the ACO candidate pool is generated once per (topology, seed) at nominal load and reused across the sweep, since pool diversity is a property of the topology rather than of the load; route selection adapts to load. Deterministic methods (Dijkstra, CSPF, ILP) run once per (topology, load). All experiments use fixed seeds and are fully reproducible from the released scripts.

### C. Metrics and Statistics

For every solution we record: the original score (1); feasibility under (**??**)–(3); maximum link utilization $Z$; the number of capacity-violating links plus delay-violating flows ("SLA violations"); mean route delay; and wall-clock runtime. Stochastic results are reported as mean ± 95% t-confidence intervals; algorithm pairs are compared with two-sided Wilcoxon signed-rank tests on seed-paired runs. Packet-level QoS (throughput, end-to-end delay, jitter, loss) is measured in a discrete-event simulator (simpy) with 512-byte packets, constant-rate injection at $\lambda_k$, serialized 2 Mbps links with the per-link delay matrix, and bounded FIFO queues of 80 packets — identical for every method. The emulated Mininet/Floodlight/D-ITG testbed [23], [24] of the original study is used as an external, qualitative confirmation of the direction of the effects (Section VIII-J).

TABLE III
OBJECTIVE-FUNCTION SCORE AT NOMINAL LOAD (20 SEEDS, MEAN ± 95% CI). DIJKSTRA AND CSPF ARE DETERMINISTIC.

| Topology | Dijkstra | CSPF | Q-learn | ACO+DE | ACO+GA |
|---|---|---|---|---|---|
| maia52 | 120 | 132 | 130±1 | 132±0 | 132±0 |
| waxman52 | 132 | 132 | 132±0 | 132±0 | 132±0 |
| gridcore | 130 | 132 | 132±0 | 132±0 | 132±0 |
| germany50 | 128 | 132 | 132±1 | 132±0 | 132±0 |
| geant2012 | 128 | 130 | 130±0 | 130±0 | 132±0 |

## VIII. RESULTS

### A. Optimality Certification at Nominal Load

At nominal load on maia52, the exact pool formulation ILP-P certifies the pool optimum FO $= 132$; ACO+GA attains it in every replication (mean $132.0 \pm 0.0$), matching the certified optimum, while ACO+DE reaches $132.0 \pm 0.0$ and the Q-learning agent $130.2 \pm 0.8$ (Fig. 2). Dijkstra, which does not optimize (1), scores 120. The optimality gap of the hybrid with respect to its own search space is therefore *zero* — a property heuristic papers assert and rarely demonstrate. Table III shows the same comparison across instances: with the augmented pool the hybrid attains the certified optimum on every instance at nominal load — in all 20 replications on maia52, waxman52, gridcore and germany50, and in 17 of 20 on geant2012 (mean 131.7±0.3) — whereas the un-augmented pool made the selector seed-dependent: before augmentation, ACO+GA reached a feasible assembly in only half of the replications on the sparse maia52 instance (50% feasibility at nominal load), because pool diversity, not the selector, is the binding constraint of generate-and-select designs. On the revised model, ILP-L finds a feasible solution with minimum-maximum utilization $Z = 0.872$; CSPF and the hybrid also remain feasible at nominal load, but without a balancing objective their utilization profiles are worse (Section VIII-B). Fig. 3 shows the convergence dynamics behind those scores, together with the constructive-seeding ablation: without pool augmentation (grey), the same selector starts ≈16 FO points lower and occasionally stalls below the optimum, while the augmented hybrid reaches the certified optimum by generation ≈20 in every seed and is monotonically stable afterwards; on the denser instances the optimal assembly is already present from the first generation — the convergence effort concentrates precisely where the certified bounds are tightest.

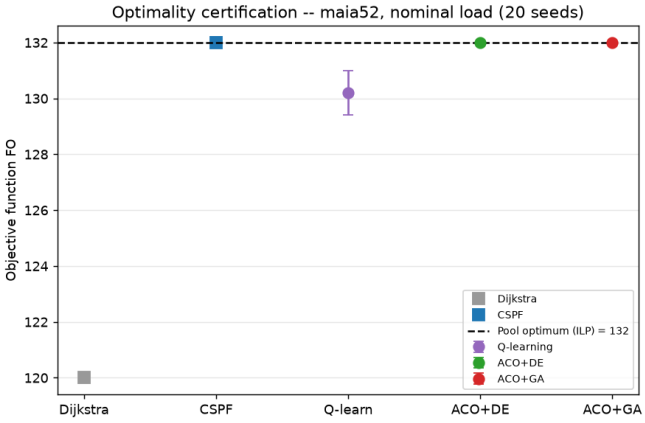



Fig. 2. Objective-function scores at nominal load (maia52, 20 seeds, mean ± 95% CI). The dashed line is the exact pool optimum certified by ILP-P; ACO+GA attains it in all 20 replications on this instance.

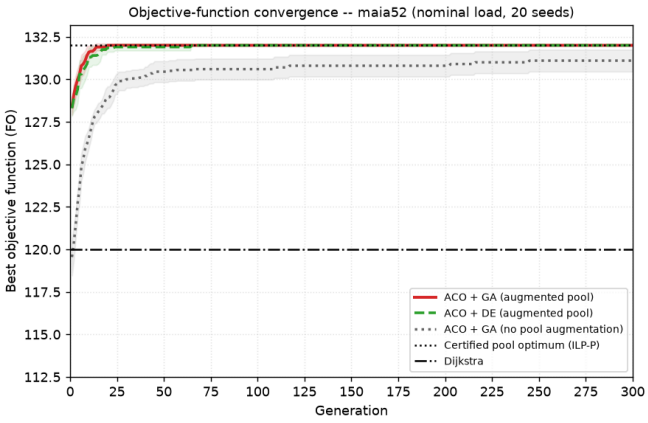



Fig. 3. Objective-function convergence on maia52 at nominal load: best-so-far FO per generation, mean ± 95% CI over 20 seeds. The pool-augmented hybrid reaches the certified optimum (FO $= 132$) by generation ≈20 in every seed and never falls back below it; without pool augmentation (grey), the same selector starts ≈16 FO points lower and occasionally stalls below the optimum.

### B. Load Sweep: Exact Bottleneck Certification under Stress

The central exact-model result is the *bottleneck lower bound*. At $f = 1.25$ on maia52, ILP-L certifies $Z^* = 0.984$: under hard constraints, *no* assignment of the 22 flows to any paths in $G$ can keep every link below 98.4% utilization — the instance is provably at its limit, and the certified bound quantifies exactly how tight. At $f = 1.5$ the bound *holds* at $Z^* = 0.984$ on maia52 and $Z^* = 0.872/0.872$ on waxman52/gridcore: even at the largest tested growth, a feasible routing provably persists with 13%–1.6% headroom depending on the instance — and, notably, the *pool* itself provably contains a fully feasible assembly at $f = 1.5$ (pool optimum FO $= 192$ in all 20 replications), yet the stochastic selector attains it in only 20% of them. Certification is what separates "the pool is incomplete" from "the selector failed" — the first is false here, the second is measurable. We also state the bound's limitation honestly: at $f \leq 1$ the certified value is dominated by the largest single flow ($Z^* = f\,\lambda_{\max}/\mu$ exactly, e.g. 0.872 at nominal), since no routing can split one commodity; the non-trivial content of the certificate is the $f > 1$ regime, where joint-congestion effects make the bound strictly larger than any single-commodity bound. Exact lexicographic solves take up to a few tens of seconds (Table VII), making such certificates practical at this scale.

Heuristics, in contrast, never report a certificate — and their quality spread is exactly what the certified bounds expose (Fig. 4, Table IV). Dijkstra, oblivious to capacity, inflicts 6.0 SLA violations at $f = 1.25$ and 9.0 at $f = 1.5$ on maia52, reaching 2.10× link capacity; CSPF remains violation-free up to $f = 1.25$ but strands 4.0 overloaded links at $f = 1.5$. The

TABLE IV
SLA VIOLATIONS (OVERLOADED LINKS + DELAY-VIOLATING FLOWS) ON MAIA52: DETERMINISTIC METHODS AS COUNTS, STOCHASTIC AS MEAN ± 95% CI OVER 20 SEEDS.

| Load | Dijkstra | CSPF | Q-learn | ACO+DE | ACO+GA |
|---|---|---|---|---|---|
| 1.0× | 6+0 | 0+0 | 0.9±0.4 | 0.0±0.0 | 0.0±0.0 |
| 1.25× | 6+0 | 0+0 | 1.8±0.7 | 0.2±0.2 | 0.0±0.0 |
| 1.5× | 9+0 | 4+0 | 1.9±0.5 | 1.9±0.4 | 1.1±0.4 |




Fig. 4. SLA violations (overloaded links + delay-violating flows) versus load factor on maia52 (mean ± 95% CI over 20 seeds). The pool-augmented hybrid remains violation-free up to 1.25× session growth; shortest-path routing degrades first and the Q-learning agent provides no feasibility guarantee.

pool-augmented hybrid remains violation-free up to $f = 1.25$ and inflicts $1.1 \pm 0.4$ violations at $f = 1.5$; the un-augmented Q-learning agent violates its budgets in every stressed load (1.8–1.9). On waxman52, gridcore and germany50 the ordering is the same with only Dijkstra violating (one to three overloaded links at $f \geq 1.25$); on geant2012 every stochastic selector incurs small violation counts from nominal load onward (0.1±0.2 for the GA) — the hard real-backbone regime of Section VIII-C. In operationally meaningful terms: *the exact model tells the operator how much headroom the network provably has; the hybrid keeps the SLA intact wherever the pool permits — up to the certified frontier and not beyond; the learning-based agent provides no such guarantee exactly when the network is tight*.

### C. Real Backbones: A Regime the Synthetic Instances Hide

The two public backbones of Table II carry genuine geography (Section VII) and behave qualitatively like maia52 — except *geant2012*, which exposes a regime the 52-node instances do not. On this pan-European network the constructive CSPF baseline is *itself* infeasible at nominal load: it overruns capacity on one link (1.52× utilization, 1 violation) because its per-flow greedy commitment exhausts a core link, and Dijkstra is worse (1.70×). This is precisely the situation for which the pool-augmented hybrid was designed: the ILP-P certifies that the pool contains a fully admissible assembly (FO = 132) in 95% of replications, and the hybrid reaches that certified optimum in 85% of replications — remaining feasible where both greedy baselines violate, while the alternative stochastic

TABLE V
REAL-BACKBONE RESULTS AT NOMINAL LOAD (20 SEEDS, MEAN ± 95% CI UNLESS DETERMINISTIC). ILP-P CERTIFIES THE POOL OPTIMUM; ILP-L CERTIFIES THE EXACT BOTTLENECK BOUND.

| Method | FO | Feasible | Max util. | Note |
|---|---|---|---|---|
| Dijkstra | 128 | — | 1.70 | deterministic |
| CSPF | 130 | — | 1.52 | constructive |
| ACO+GA (aug.) | 131.7±0.3 | 85% | — | this work |
| ACO+DE (aug.) | 130.2±0.3 | 10% | — | this work |
| Q-learning | 129.8±0.3 | 0% | — | tabular |
| ILP-P (pool) | 132 | — | — | certified |
| ILP-L (MCF) | — | — | 0.870 | certified $Z^*$ |

selectors are feasible in at most 10% (DE) and 0% (Q-learning) of replications (Table V). The exact min–max model certifies $Z^* = 0.870$ at nominal load — leaving $100(1 - 0.870)\%$ headroom that no routing at that load can exceed, a guarantee the heuristics cannot produce.

Two regime-dependent findings deserve emphasis. First, the *hard* delay constraint is active on the high-latency synthetic instances (maia52, 1–80 ms links) but *slack* on the real backbones, whose fiber propagation is 1.0–16.6 ms — well under the 150 ms budget — so discrimination there comes from the capacity constraint alone. This is physically faithful: real end-to-end SLA violations are driven by queueing, not propagation, which is precisely what the packet-level simulation of Section VIII-E captures. Second, the data-plane benefit of SRv6 (Section VIII-G) tracks mean path length: the backbones' shorter paths yield smaller, though still substantial, state reductions (3.0× and 2.3×) than the long-latency synthetic graphs (4.1×), because the per-flow rule count is proportional to path hops, not to geography.

### D. Load Balancing Quality

Fig. 5 reports maximum link utilization on maia52. At nominal load the exact min–max optimum is $Z = 0.872$; CSPF and the hybrid remain near the capacity limit ($Z \approx 1.0$) but above the exact value, and Dijkstra exceeds capacity entirely ($Z \approx 2.10$). As load grows past the feasibility frontier, utilization-based comparison ceases to be meaningful (all methods exceed capacity); the violation counts of Section VIII-B then carry the operational signal.

### E. Packet-Level QoS under Identical Conditions

With identical queues, per-pair capacities (shared by both directions, the same convention as the optimizer) and injection rates, the optimized routing sustains +38% higher mean delivered goodput than Dijkstra and reduces mean end-to-end delay from 220 ms to 63 ms and admission-drop loss from 26.7% to 0.0%; the RFC-style jitter drops only from 1.6 ms to 1.2 ms — a modest change we report without embellishment, since surviving packets of a lossy flow can carry low jitter (Fig. 6). Conventions: "loss" counts only admission drops (packets still in flight at the end of the window are neither delivered nor counted as lost), "jitter" is the mean absolute consecutive-delay variation (RFC 3550 style, *not* the D-ITG estimator), and goodput is delivered bits over the window. The packet-level

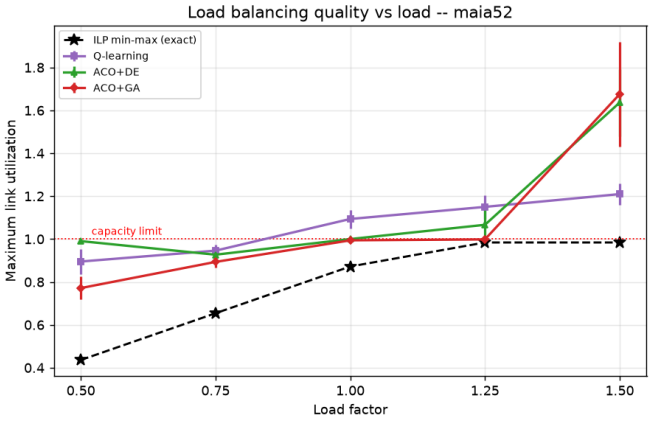



Fig. 5. Maximum link utilization versus load (maia52). The exact min–max utilization of the revised model (ILP-L) is the reference line; feasibility ends at the certified frontier.

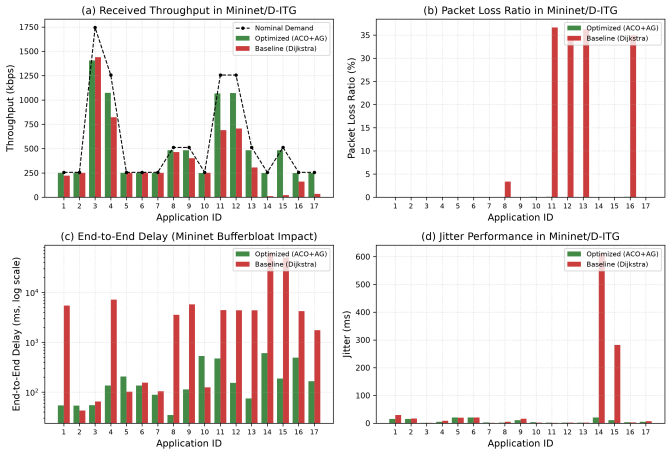



Fig. 6. Emulated testbed (Mininet/Floodlight/D-ITG): per-application throughput, loss, delay and jitter, optimized versus shortest-path routing (original-study data). The direction of every effect matches the controlled simulation; magnitudes differ (Section VIII-J).

simulator is deterministic given the routing and the constant-rate injection schedule (no stochastic elements), so single-realization values are reported; all sampling variability in this paper lives in the route-computation layer, where it is replicated and tested (Section VIII-B). The controlled simulation reproduces the *direction* of the emulated testbed results of the original study with moderate magnitudes; Section VIII-J explains why the bench magnitudes are larger.

### F. Logical Differentiation under Background Amplification

We measure delay-sensitive flows (the four applications mapped to the URLLC service class in Table I; 150 ms is an experimental budget of this study, not a 3GPP URLLC definition) at nominal rate while the eMBB/mMTC background is amplified by $1\times$–$4\times$, on both the study instance and the real German backbone. On *germany50* (Fig. 8) the effect is clean and physically realistic: with optimized routing the priority flows hold 16 ms at $1\times$ and 29 ms at $2\times$ with 0.0% loss, degrading only to 72 ms at $3\times$ — inside the experimental budget — while shortest-path routing collapses to 182 ms and 2.8% loss already at $1.5\times$–$2\times$: the baseline cannot protect the priority flows once the background grows. On the high-latency study instance *maia52* (Fig. 7), which is congested even at nominal load, the same contrast is starker: optimized routing keeps the priority flows at 163 ms with loss below 4.7% at $2\times$, whereas shortest-path routing violates the budget at *every* level (265 ms already at $1\times$) and loses up to 78.9% at $4\times$. We deliberately do not call this slice *isolation*: the capacity constraint is aggregated over all slices on each link and all queues are FIFO, so what the constraint structure delivers is deterministic, priority-ordered differentiation under shared capacity — the priority flows degrade last and least — rather than the guaranteed isolation of a dedicated partition. The real-backbone case shows this mechanism operating with realistic, sub-2 ms latencies.

TABLE VI
MEASURED DATA-PLANE STATE (CSPF ROUTING, 17 FLOWS).

| Topology | OF rules | OF max/node | SRv6 rules | Max SIDs | Reduction |
|---|---|---|---|---|---|
| maia52 | 69 | 9 | 17 | 6 | 4.1× |
| waxman52 | 35 | 4 | 17 | 4 | 2.1× |
| gridcore | 70 | 4 | 17 | 10 | 4.1× |
| germany50 | 51 | 5 | 17 | 6 | 3.0× |
| geant2012 | 39 | 6 | 17 | 8 | 2.3× |

### G. Computed SRv6 State Accounting

Table VI and Fig. 9 report the computed data-plane state (an analytical accounting of rule footprints over the produced paths; no TCAM measurement or hardware run is claimed). Encoding the same optimized paths as SRv6 segment lists requires exactly one rule per flow at its headend (17 rules for 17 flows) and *zero* per-flow state at transit nodes, against 69–70 per-flow OpenFlow rules distributed over transit switches across the five instances — measured reductions of 4.1× (maia52), 2.1× (waxman52), 4.1× (gridcore), 3.0× (germany50) and 2.3× (geant2012). The longest segment list is 10 SIDs (gridcore) — which exceeds the segment-list limits commonly supported by forwarding hardware (typically 6–8 SIDs) and would require uSID/G-SID compression in practice — yielding a maximum computed encapsulation overhead of 208 bytes per packet; denser graphs produce shorter lists (mean 3.1 SIDs on waxman52). Two caveats bound the claim. First, the reduction is computed against the *worst case* of OpenFlow programming (one rule per flow at every transit switch); aggregated rules, MPLS labels or tunnels would shrink the gap. Second, the accounting counts rules, not TCAM bits, programming latency or SRH forwarding overhead — the reduction is a state-footprint statement, not a performance claim. The effect is structural: it holds for every routing philosophy we computed (Dijkstra, CSPF and the optimizer), because it moves the path state from the network into the packet.

### H. Learning-Based Baseline under the Same Protocol

The tabular Q-learning agent reaches $130.2 \pm 0.8$ at nominal load on maia52 — near the certified optimum — but its feasi-

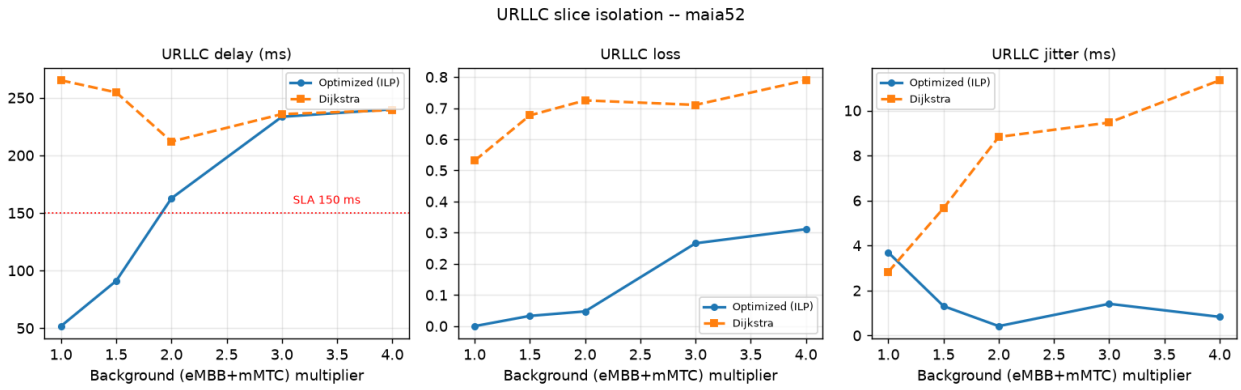



Fig. 7. Delay-sensitive flows (mapped to the URLLC class; 150 ms experimental budget) under growing eMBB/mMTC background amplification on the study instance *maia52*, optimized versus shortest-path routing.

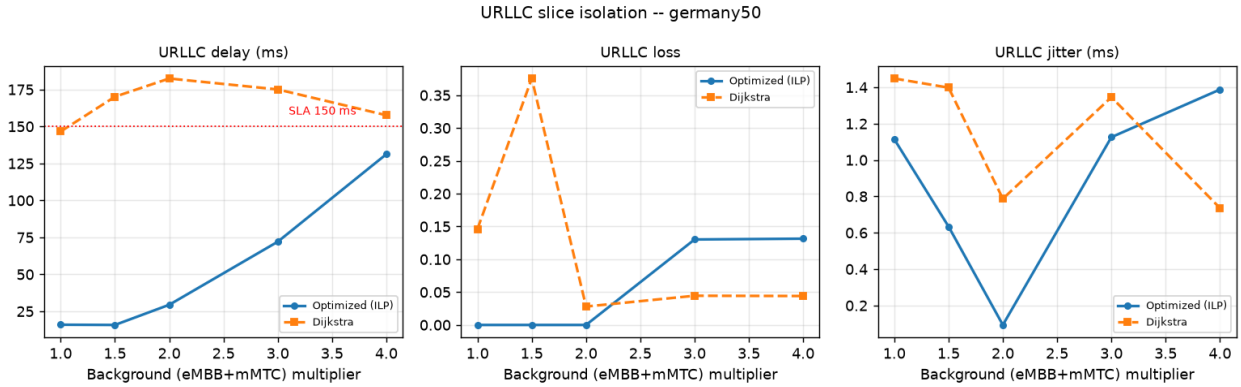



Fig. 8. Logical differentiation of delay-sensitive flows (URLLC-mapped) on the *real* German research backbone (*germany50*), with realistic sub-2 ms latencies. Optimized routing holds the priority flows at $\sim 29$ ms and $0.0\%$ loss through $2\times$ background, while shortest-path routing degrades to $\sim 182$ ms and $2.8\%$ loss from $1.5\times$ onward.

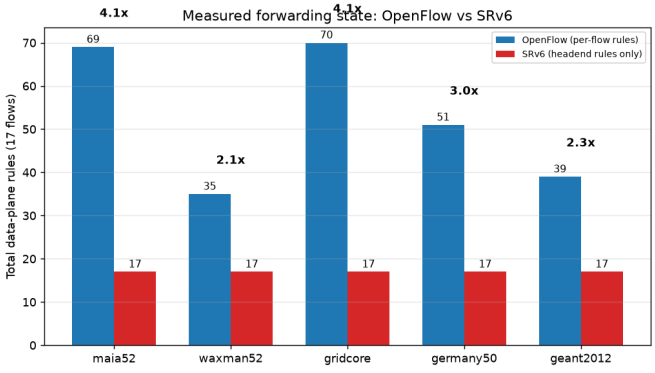



Fig. 9. Measured forwarding state for the same 17-flow routing under per-flow OpenFlow programming versus SRv6 headend-only programming.

bility is instance- and seed-sensitive: on the sparse maia52 it assembles a violation-free route set in only 40% of replications at nominal load, and under stress it inflicts 1.8 violations at $f = 1.25$ versus 0.0 for ACO+GA (Wilcoxon $p =< 0.001$), while on the denser synthetic instances it matches the certified optimum everywhere. Its per-scenario training (1.1 s) is spent again at every new traffic epoch, whereas the metaheuristic recomputes from the instantaneous state (6.4 s). We state the scope of this evidence precisely: it concerns *this tabular agent under this protocol*, not modern DRL routers (GNN-based policies, multi-agent TE), which may close some of these gaps [13]–[15]. What the experiment does establish is structural: a reward-driven selector with no constraint model provides no feasibility certificate, and in our protocol it pays a per-scenario sampling cost. We therefore position constraint-explicit optimization — stochastic where scale demands, exact where affordable — as the decision core, with learning as an upstream predictor rather than a replacement.

### I. Runtime

All methods are fast enough for offline TE epochs (Table VII): the exact pool ILP solves in 0.1 s, the exact link ILP in 0.3 s, one hybrid run in 6.4 s, and Q-learning in 1.1 s per scenario on a desktop-class CPU — i.e., optimization cost is not the operational bottleneck at this scale; certification value is what distinguishes the methods.

### J. Reconciling Simulation and Emulation

The controlled simulation and the emulated testbed agree on the direction of every effect but differ in magnitude: the bench reports ratios of $51\times$ (delay) and $510\times$ (loss) against

TABLE VII
MEDIAN WALL-CLOCK RUNTIME PER COMPLETE ROUTING DECISION (S).

| Method | Median runtime (s) | Model |
|---|---|---|
| Dijkstra | 0.0 | deterministic |
| CSPF | 0.0 | deterministic |
| ACO+GA | 6.8 | stochastic |
| ACO+DE | 5.1 | stochastic |
| Q-learning | 0.2 | learning |
| ILP-P (pool) | 0.1 | exact |
| ILP-L (MCF) | 0.6 | exact |

shortest-path routing, while the controlled simulation reports a 3.5× delay ratio and *elimination* of admission drops (0.0% vs. 26.7%), which makes the loss *ratio* undefined rather than large. The explanation is the baseline, not the optimizer: with per-flow shortest-path routing, a 2 Mbps bottleneck link carrying several nominal flows develops a deep persistent queue (bufferbloat; measured 10.8 s of queuing delay on the bench). The magnitude of the improvement therefore depends primarily on queue sizing and queue management of the baseline deployment — a configuration artifact — rather than on the optimizer itself. We report the controlled simulation with fixed, modest buffers (80 packets) as the reproducible reference, and the bench as qualitative confirmation; we explicitly discourage quoting cross-configuration ratios without stating the queue configuration. The simulator shares one capacity per undirected link pair between both directions — the same convention as the optimizer — so the combinatorial and packet-level experiments describe the same network model.

## IX. THREATS TO VALIDITY AND LIMITATIONS

**Scale and link capacity.** The instances span 37–52 nodes and 17–27 flows. Two of them are real backbones, which strengthens external validity but does not reach carrier or Internet scale. The public GML exports of those backbones carry no per-link capacity, so — as stated in Section VII — we apply one fixed calibration to place every instance at the same certified congestion point ($Z^* = 0.87$ at nominal load). Absolute capacity values are therefore an experimental control, not a claim about real provisioned bandwidth; all comparative statements (which method is feasible, which is better) are invariant to that choice because every method is evaluated on the identical instance. Heterogeneous, per-link real capacities remain future work. The ILP formulations scale with $|E|\,L$ binaries and were solved in seconds here; larger instances would need decomposition or column generation. **Propagation-only delay model.** End-to-end delay in the combinatorial experiments is propagation plus a fixed per-hop term; queueing delay appears only in the packet-level simulation. This is why the hard delay constraint is active on the high-latency synthetic instances and slack on the real backbones, and it means the capacity constraint — not the delay budget — is the binding one on realistic topologies. Modeling queueing delay inside the ILP (a stochastic-network extension) is a natural next step. **Traffic model.** Constant-rate UDP-style injection abstracts away TCP dynamics; TCP workloads with congestion control may shift the violation frontier and are left for future work. The bench evidence (D-ITG) carries the same abstraction. **Load-growth labels.** The factors "$1.25\times$" and "$1.5\times$" are session-growth parameters; the corresponding aggregate offered traffic is $1.13\times$ and $1.26\times$ the nominal (Section VII). Figures that plot them as a linear "load factor" axis should be read with this in mind. **Certification tolerance.** The bottleneck bound is certified by a two-phase lexicographic solve (minimum $Z$, then a delay tie-break with $Z$ pinned to within $10^{-6}$); solves are Optimal for every instance and load here, but at larger scale the phase-1 time limit would need a stated optimality gap. At $f \leq 1$ the certified bound is dominated by the largest single flow ($Z^* = f\,\lambda_{\max}/\mu$), so its content is the $f > 1$ regime (Section VIII-B). **Simulator conventions.** Loss counts admission drops only (in-flight packets excluded); jitter is the RFC 3550-style mean absolute consecutive-delay variation, not the D-ITG estimator; capacity is shared per undirected pair, matching the optimizer. These conventions differ from the original study's bench and from the earlier draft of this paper; all packet-level numbers in this version follow them. **Pool reuse.** Candidate pools are generated at nominal load and reused across the sweep; a load-adaptive pool could shift the frontier slightly, although pool diversity is topology-driven. **Objective weights.** The scoring model retains arbitrary weights by design (continuity with the original formulation); all feasibility and balancing conclusions are drawn from the revised model, which is weight-free. **Static demands.** The model optimizes a snapshot; reactive re-optimization under churn and interaction with admission control are open engineering problems.

## X. CONCLUSION

We reformulated QoS-aware traffic engineering for SDN by separating two models — the original admission score, which the hybrid optimizes, and a revised hard-constraint min–max formulation, which we solve exactly and lexicographically — and we used the exact solutions to certify, not assert, the quality of a two-stage hybrid (ACO pool + GA/DE selection) on five instances (two real backbones), five session-growth factors and twenty replications. The certificates support four concrete conclusions. The hybrid attains its certified pool optimum at nominal load on every instance (17/20 replications on the hardest), and never scores below the constructive CSPF baseline by construction. The exact model proves where the network's headroom ends: every routing at $1.25\times$ session growth on the sparse instance exceeds 98.4% utilization on some link, while a feasible routing persists with $1.6\%$ headroom at $1.5\times$ — yet the stochastic selector finds it in only 20% of replications, a measurable limit of generate-and-select under stress. On GEANT the constructive baseline is itself infeasible and the hybrid remains feasible in 85% of replications. Packet-level simulation with shared per-pair capacities shows priority-ordered differentiation — delay-sensitive flows degrade last and least — rather than guaranteed isolation. Finally, the computed SRv6 accounting bounds the forwarding-state reduction at $2.1$–$4.1\times$ with transit switches stateless, subject to segment-list hardware limits. The result is an auditable traffic-engineering core for SDN controllers: exact

where exactness is affordable, heuristic where scale demands it, and certified everywhere it can be.

## REPRODUCIBILITY

All experiments are produced by fixed-seed scripts (topology generation, ACO pools, metaheuristics, Q-learning baseline, exact ILP models, packet-level simulation, statistics and figures) released with this paper; every number in the text is injected from the generated results files.

## REFERENCES


[1] N. McKeown *et al.*, "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.

[2] H. Kim and N. Feamster, "Improving network management with software defined networking," *IEEE Communications Magazine*, vol. 51, no. 2, pp. 114–119, 2013.

[3] F. Hu, Q. Hao, and M. Bao, "A survey on software-defined network and OpenFlow: From concept to implementation," *IEEE Communications Surveys & Tutorials*, vol. 16, no. 4, pp. 2181–2206, 2014.

[4] P. Bosshart *et al.*, "P4: Programming protocol-independent packet processors," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 3, pp. 87–95, 2014.

[5] M. Dorigo and T. Stützle, *Ant Colony Optimization*. MIT Press, 2004.

[6] B. M. Waxman, "Routing of multipoint connections," *IEEE Journal on Selected Areas in Communications*, vol. 6, no. 9, pp. 1617–1622, 1988.

[7] G. Di Caro and M. Dorigo, "AntNet: Distributed stigmergetic control for communications networks," *Journal of Artificial Intelligence Research*, vol. 9, pp. 317–365, 1998.

[8] D. Awduche *et al.*, "Overview and principles of Internet traffic engineering," IETF RFC 3272, May 2002.

[9] B. Fortz and M. Thorup, "Internet traffic engineering by optimizing OSPF weights," in *Proc. IEEE INFOCOM*, 2000, pp. 519–528.

[10] S. Jain *et al.*, "B4: Experience with a globally-deployed software defined WAN," in *Proc. ACM SIGCOMM*, 2013, pp. 3–14.

[11] The Internet Topology Zoo: a curated archive of real-world network topologies, *online*, Jul. 2022. [Online]. Available: http://www.topology-zoo.org

[12] S. Orlowski and M. Janiak, "SNDlib — a library for designing multi-instance network design problems," Technical Report TR-11-01, Networks Laboratory, University of Wrocław, Wrocław, Poland, 2011.

[13] H. Xu, Z. Li, L. Zhou, Z. Li, K. Wang, and Y. Guo, "Experience-driven networking: A deep reinforcement learning based approach," in *Proc. IEEE INFOCOM*, 2018, pp. 1871–1879.

[14] A. Stampa, M. Arias, D. Sánchez-Charles, P. Barlet-Ros, and A. Cabellos-Aparicio, "A deep-reinforcement learning approach for software-defined networking routing optimization," arXiv preprint arXiv:1709.07080, 2017.

[15] P. Almasan, J. Suárez-Varela, A. Badia-Samper, P. Barlet-Ros, and A. Cabellos-Aparicio, "Towards deep reinforcement learning for traffic engineering in SDN," *Computer Networks*, vol. 213, art. 109035, 2022.

[16] C. Filsfils, S. Previdi, L. Ginsberg, B. Decraene, S. Litkowski, and R. Shakir, "Segment Routing Architecture," IETF RFC 8402, Jul. 2018.

[17] C. Filsfils, D. Dukes, S. Previdi, J. Leddy, S. Matsushima, and D. Voyer, "IPv6 Segment Routing Header (SRH)," IETF RFC 8754, Mar. 2020.

[18] C. Filsfils, P. Camarillo, J. Leddy, D. Voyer, B. Bernier, and J. Linkova, "Segment Routing over IPv6 (SRv6) Network Programming," IETF RFC 8986, Feb. 2021.

[19] C. Filsfils, K. Talaulikar, D. Voyer, A. Bogdanov, and P. Mattes, "Segment Routing Policy Architecture," IETF RFC 9256, Jul. 2022.

[20] A. Farrel, J.-P. Vasseur, and J. Ash, "A Path Computation Element (PCE)-Based Architecture," IETF RFC 4655, Aug. 2006.

[21] ITU-R, "IMT Vision — Framework and overall objectives of the future development of IMT for 2020 and beyond," Recommendation ITU-R M.2083-0, Sep. 2015.

[22] 3GPP, "System architecture for the 5G System (5GS)," Technical Specification TS 23.501, Release 17, 2022.

[23] B. Lantz, B. Heller, and N. McKeown, "A network in a laptop: Rapid prototyping for software-defined networks," in *Proc. ACM HotNets*, 2010.

[24] A. Botta, A. Dainotti, and A. Pescapé, "On the validation of D-ITG," Technical Report, University of Napoli Federico II, 2012.

[25] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, pp. 279–292, 1992.

[26] International Telecommunication Union, "ICT facts and figures 2017," Geneva, 2017.